

Intro to Machine Learning

EECS 492

Introduction to Artificial
Intelligence

Reminders

- **Ethics**
- **HW 1 to be released Friday**
 - **Start early!**
 - **Topics: Agents and ML**
 - **Coding resources available in Canvas**
 - **Files > Coding Resources**
 - **Includes video guide on virtual environments**
- **HW1 due date: 10/6**

Today's Class

- Unsupervised learning
- Supervised learning

Learning

- How can we tell if an agent is **learning**?
 - Improves performance on future tasks after making observations about the world
- Why is learning desirable?
 - Can designers anticipate all possible situations?
 - Can designers anticipate change?
 - What if we, as people, have no idea how to structure the task?

Types of Feedback

- Reinforcement learning

- Agent learns from series of reinforcements (rewards/punishments)
- Builds off of the MDP/POMDP formalism

- Unsupervised learning

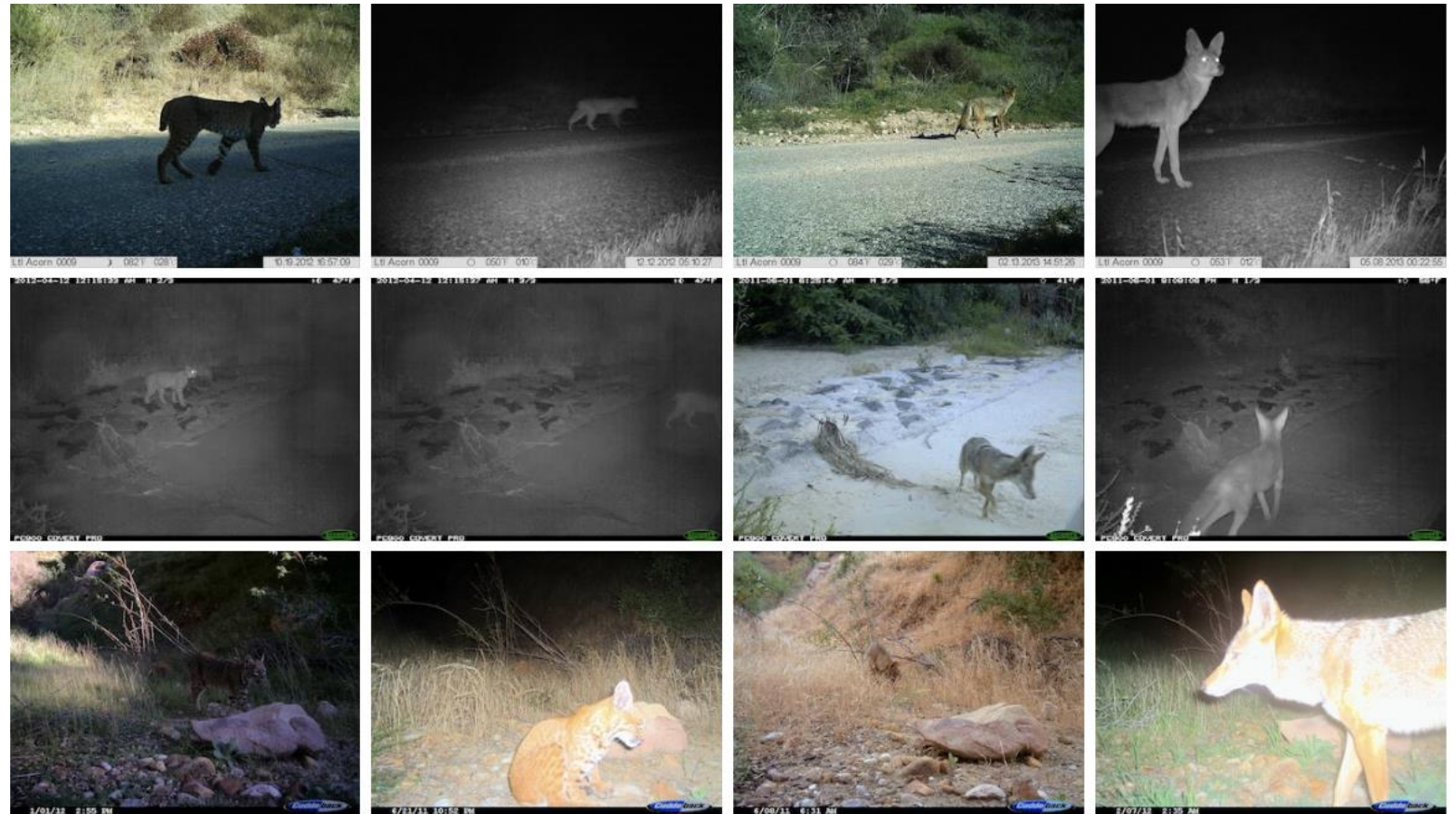
- No explicit feedback is supplied
- Most common method: clustering

**Also very popular:
Self-supervised learning!**

- Supervised learning

- Agent observes example input-output pairs
- Learns functions mapping input to output

Camera Traps



Megadetector

- Robust and generalizeable animal detection
- Reliably filtering out empty images (possibly 70-90% of the data at a sensor)
- Provided free of charge by Microsoft from 2019-2021, integrated into camera trap data management tools
- Led to large-scale adoption with over 60 NGOs, governmental agencies, and research groups using the model globally
- MegaDetector team works with organizations to check accuracy and set operating points for model confidence based on the risks of false positives vs false negatives

User experience

- Uploads images to be processed
- Verification and categorization through camera trap management software
- Process up to 10K images a day by himself with the model in the loop

How are **images** processed?



How can we **learn** to do this?



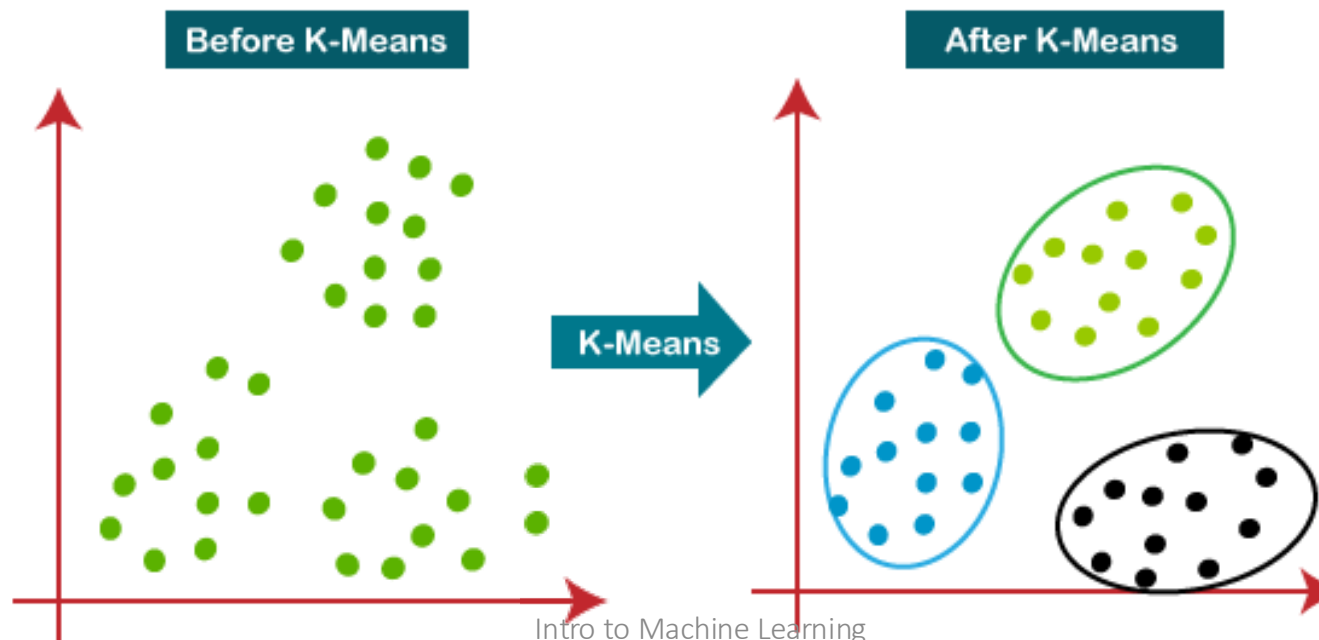
Unsupervised Learning

Unsupervised Learning

- **Unsupervised learning** – agent learns patterns in the input with no explicit feedback
- Most common task is **clustering** – detecting potentially useful clusters of input examples
- Example: given millions of images from the Internet, a system can identify large clusters of similar images
 - E.g., Maybe it will find a large cluster of images that look like cats

k-means Clustering

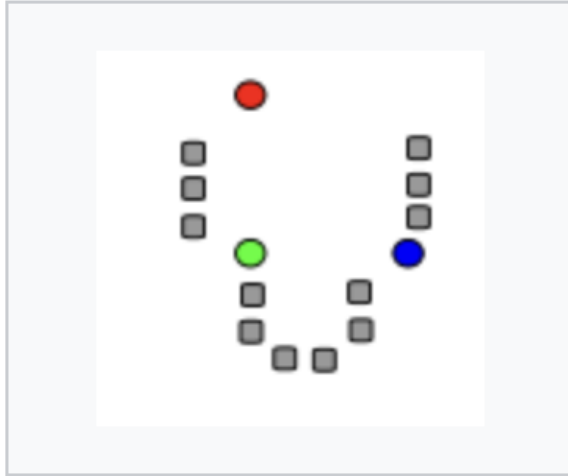
- Let's say we have n observations (data points) that we want to group into k clusters
 - Each cluster has a centroid – the mean point of the cluster
 - We want each data point to belong to the cluster with the nearest centroid



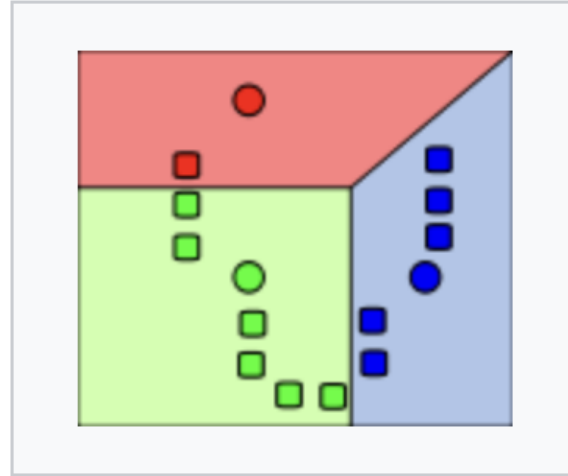
k-means Clustering Algorithm

- Lloyd's algorithm
- Uses iterative refinement
- Start by randomly choosing k points to be centroids
- Then iterate:
 - Assign each data point to the cluster with the nearest centroid
 - Now, recalculate each centroid (take the average of all the points in the cluster)
- Stop iterating when the cluster assignments no longer change

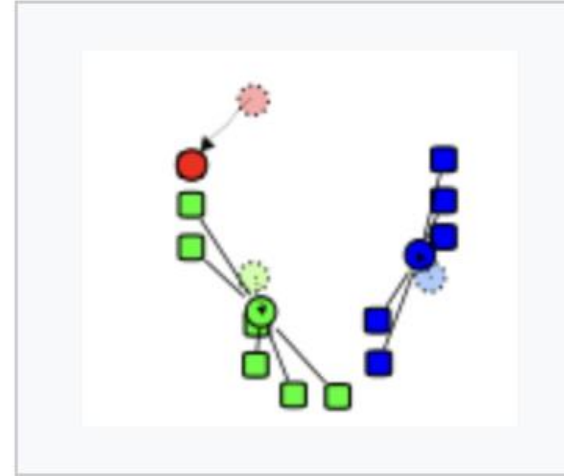
k-means Clustering Algorithm



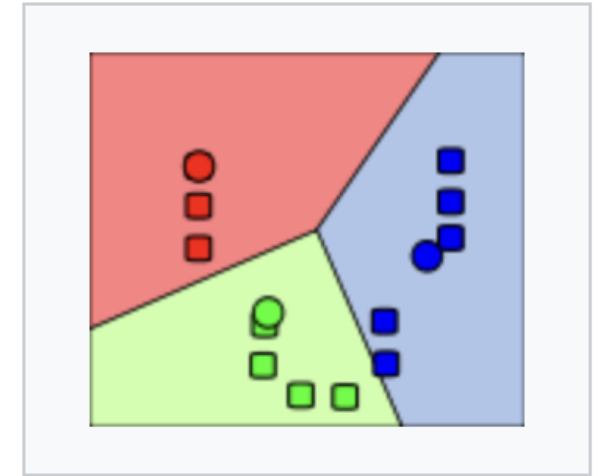
Randomly choose
3 points to be
centroids



Assign each data
point to the
cluster with the
closest centroid

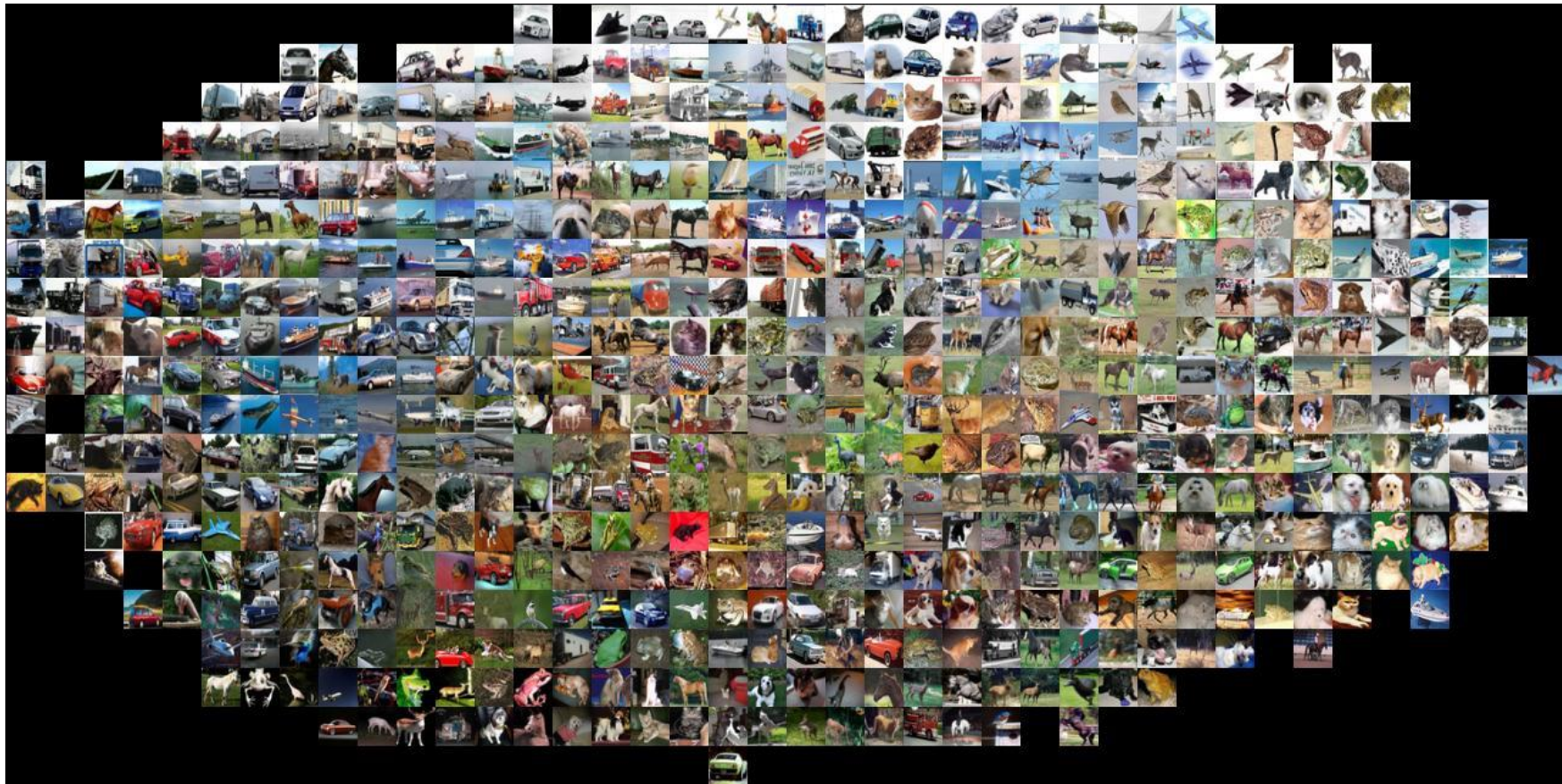
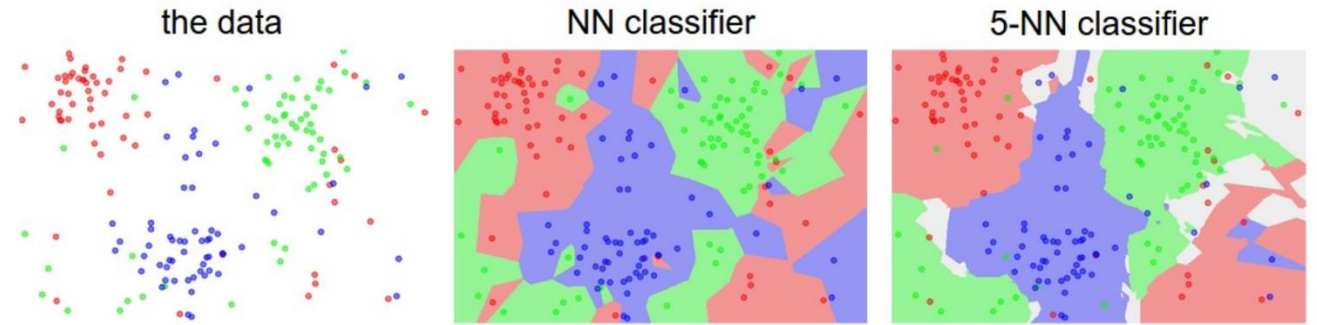


Update the
centroids



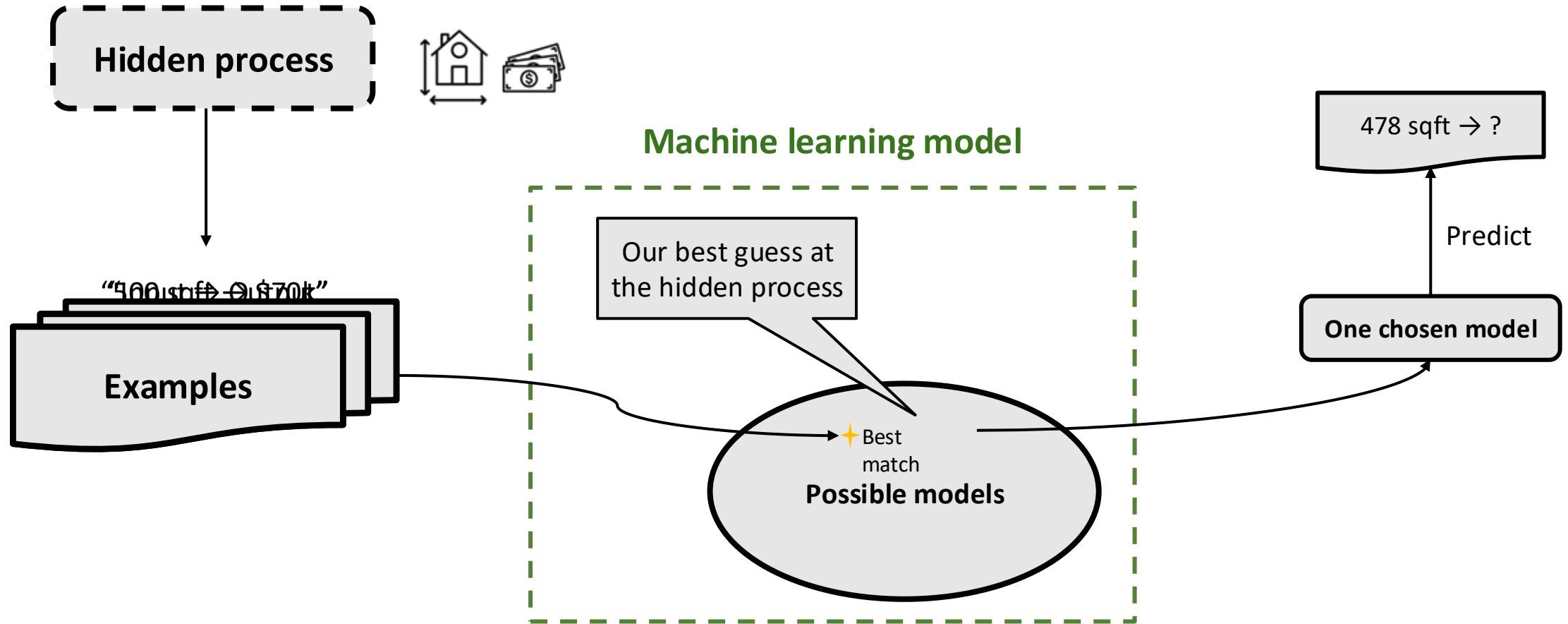
Repeat until
convergence

K-Nearest Neighbor

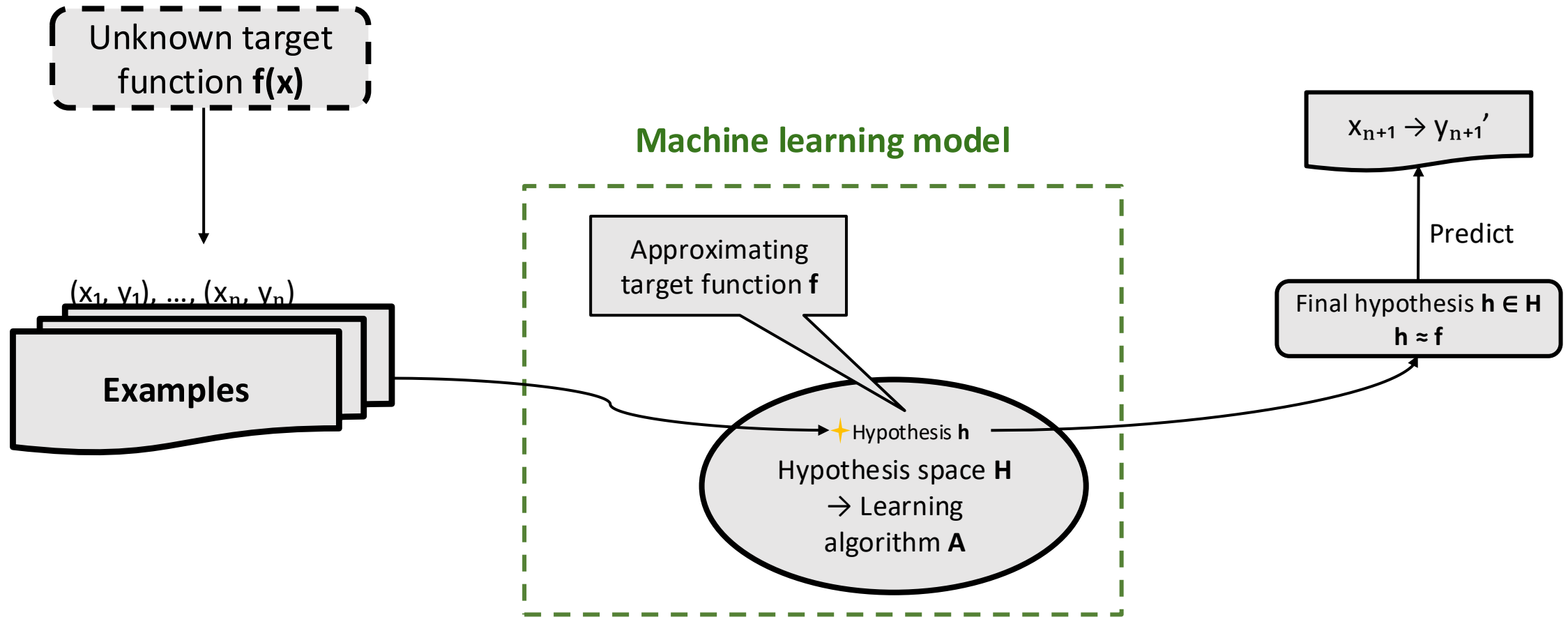


Supervised Learning

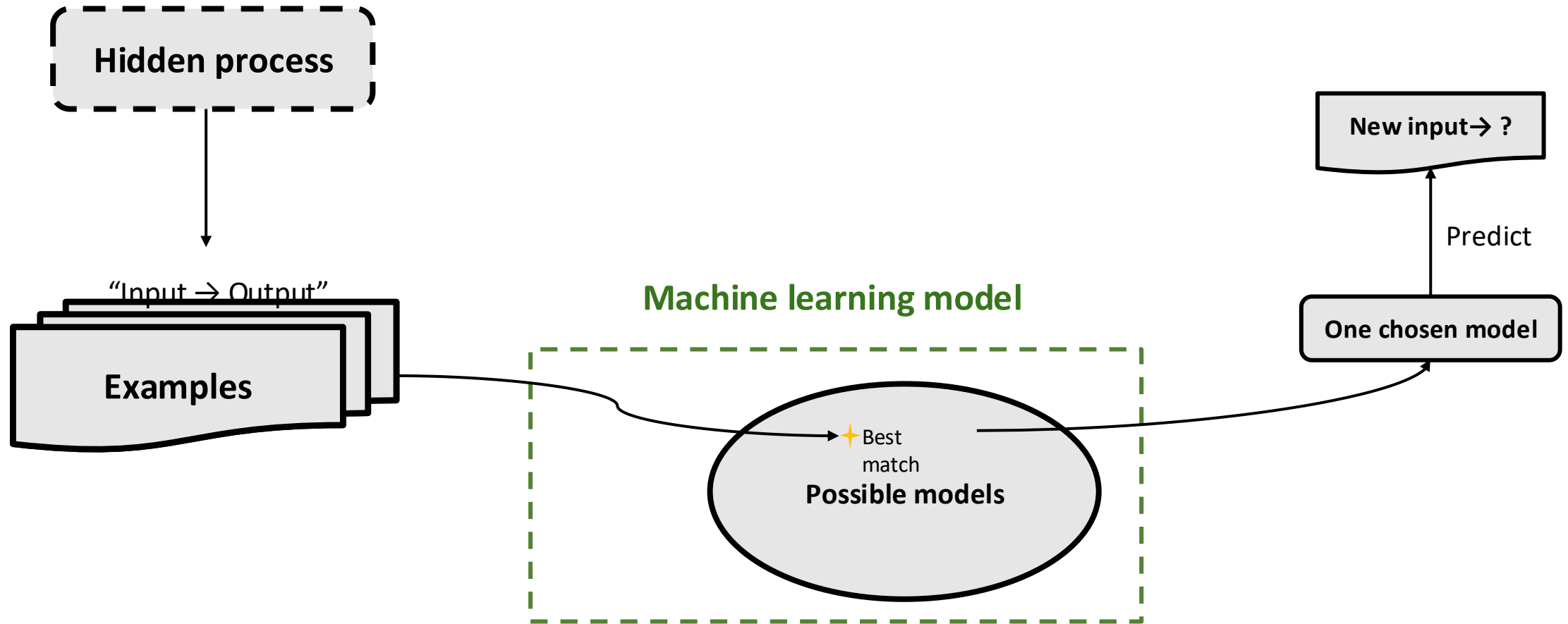
Supervised Learning



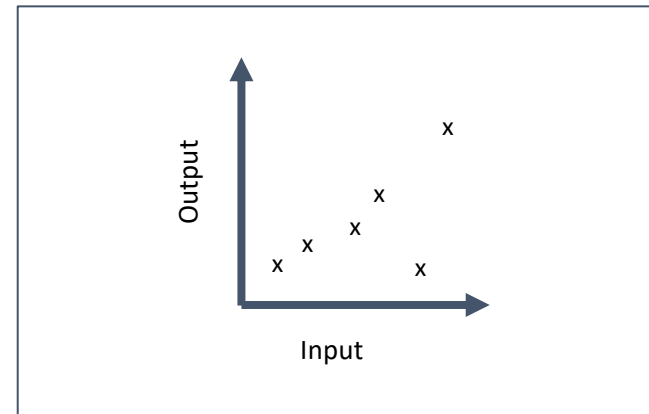
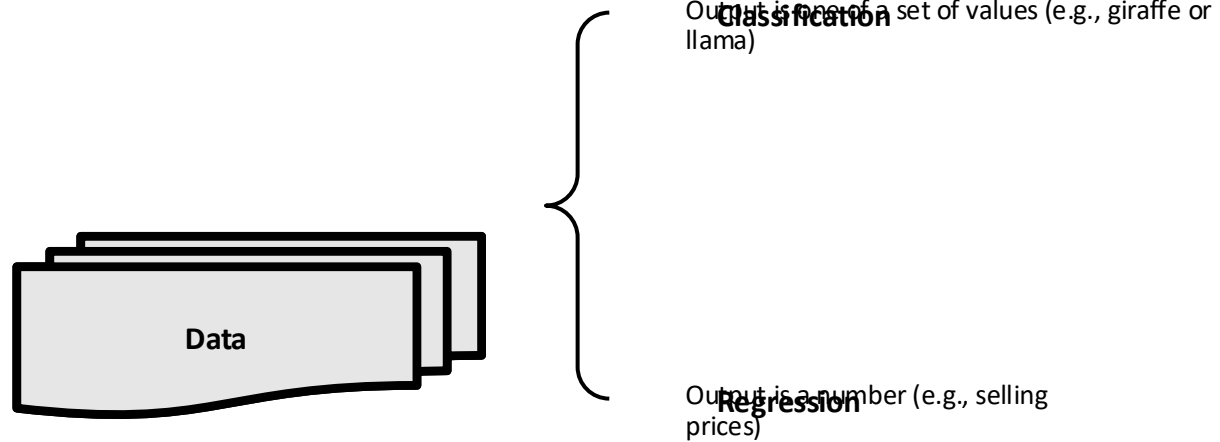
Supervised Learning (Math)



Supervised Learning



Supervised Learning



Megadetector: working towards classification of animal/no animal



Building a ML Model

- **Training Time**

- Use labeled input/output examples to learn a machine learning model
 - E.g., learn parameters of model

- **Deployment Time**

- Run trained model on examples where we don't know the output

Validating Models

- **Data separation**

- Training set – used to learn the models
- Validation set – used to adjust parameters of the models
- Testing set – used to validate the models
- Method performs well if models **generalize** to testing set



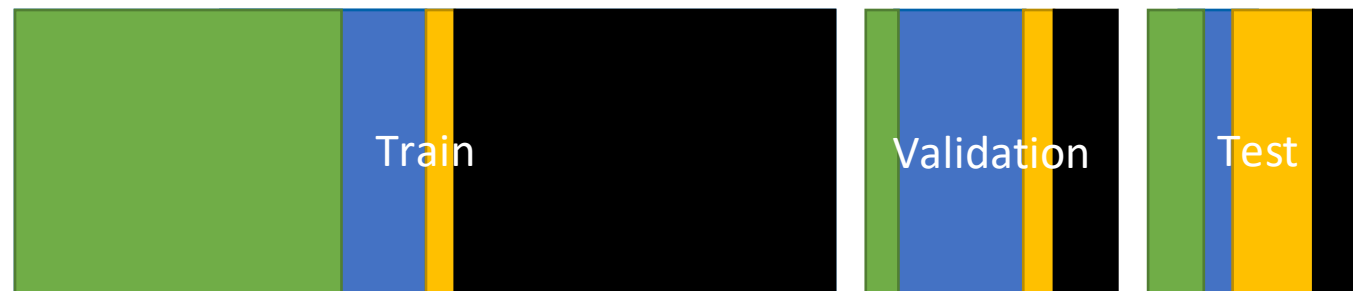
Train/validation/test



Key:



Train/validation/test



Key:



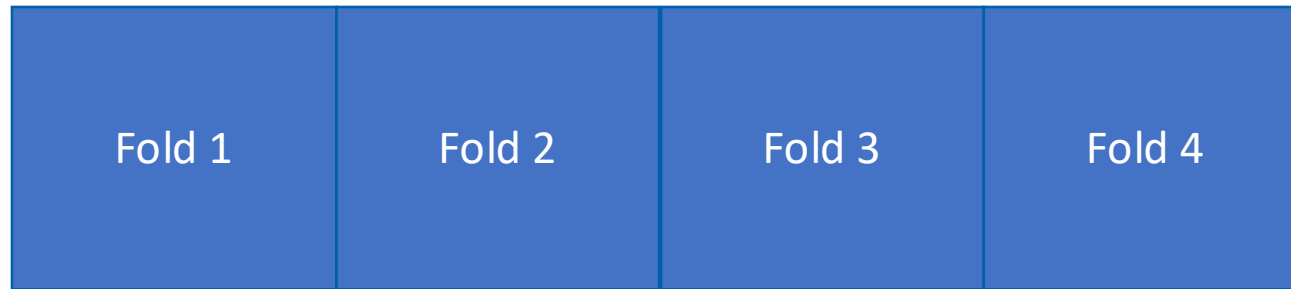
k-Fold Cross Validation



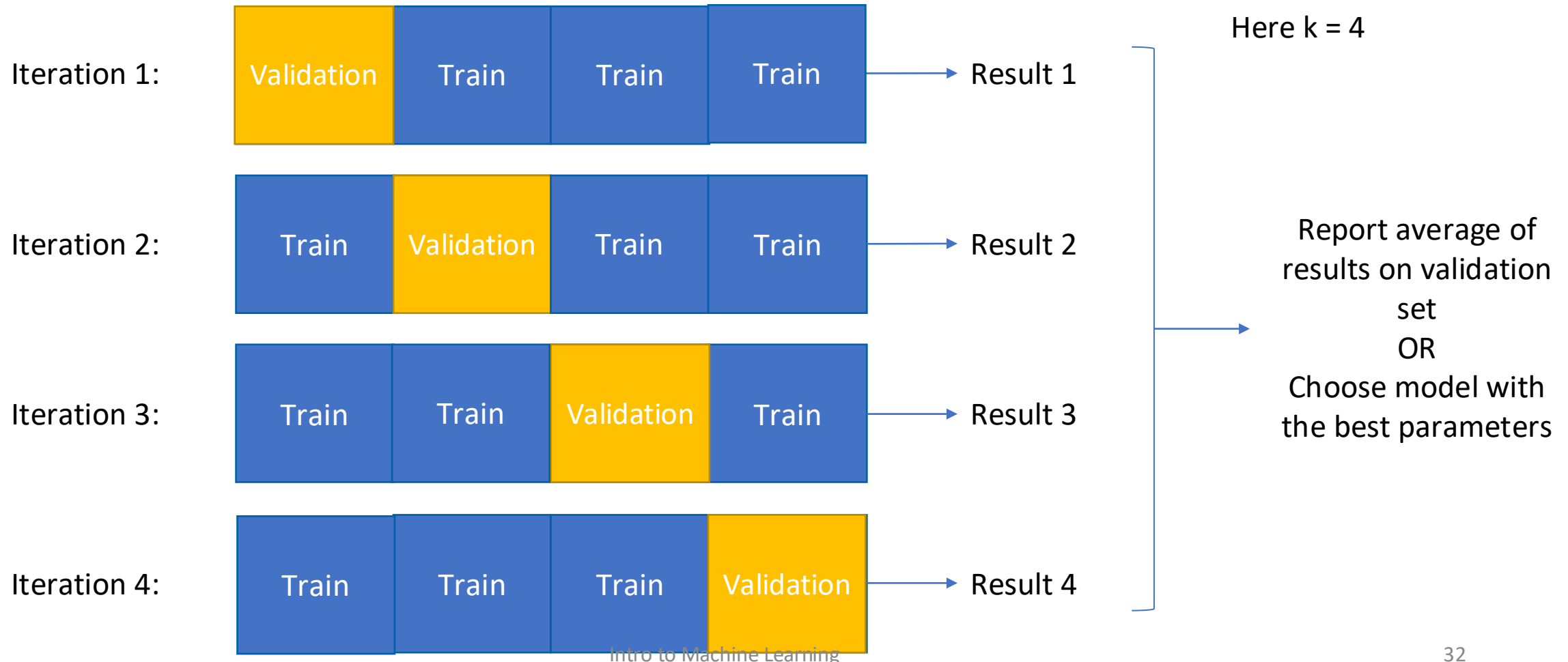
Train + Validation Data

k-Fold Cross Validation

Here $k = 4$



k-Fold Cross Validation



Bias-Variance Trade-off

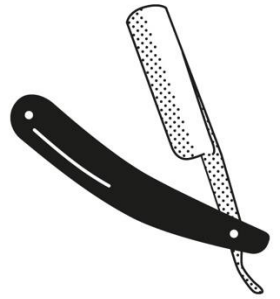
- **Bias** – (loosely) how much error the model has on the training data
- **Variance** – how much the model changes due to fluctuations in the training data

Bias-Variance Trade-off



Bias-Variance Trade-off

- Trade-off:
 - More complex (high variance) models that fit the training data well (low bias)
 - Simpler (low variance) models that don't fit the training data as well (high bias)
 - Usually can't have both low bias and low variance!
- **Ockham's razor**: choose the simplest model that matches the data
 - Simplicity alone doesn't guarantee correctness, but it often leads to more useful and understandable solutions
 - Ockham's razor promotes simplicity as a heuristic guide for model selection
- But watch out for underfitting and overfitting
 - **Underfitting** – model fails to find a pattern in the data
 - **Overfitting** – model pays too much attention to a particular dataset and can't generalize to unseen data



Megadetector

